Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра оптоелектроніки та інформаційних технологій

ЗВІТ

з лабораторної роботи № 6
з дисципліни «Основи програмування»
«Цикли»

Виконав:
студент групи ФЕП-13
Марцінко Н. Р.

Перевірив:
асистент Васюта В. М.

Львів – 2026

Мета роботи

Вивчити поняття і застосування циклів: оператори for, while, do/while, break та continue.

1. Теоретичні відомості (коротко)

Цикл for — цикл з передумовою: ініціалізація, перевірка умови, модифікація лічильника. while також перевіряє умову на початку ітерації. do/while виконує тіло принаймні один раз і перевіряє умову в кінці. break примусово завершує цикл (або switch). continue пропускає залишок поточної ітерації і переходить до наступної.

2. Завдання 1. Підрахунок голосних і приголосних у реченні (цикл for)

Програма зчитує рядок (підтримка кирилиці через wchar_t і локаль UTF-8), проходить символи циклом for, переводить у нижній регістр і перевіряє належність до списків голосних/приголосних через wcschr.

2.1. Текст програми (task1.c)

Martsinko > Lab6 > task1.c

```c
#include <stdio.h>
#include <wctype.h>
#include <locale.h>
#include <wchar.h>

int main() {
    if (setlocale(LC_ALL, "") == NULL) {
        setlocale(LC_ALL, "C.UTF-8");
    }
    wprintf(L"Введіть речення: ");
    wchar_t sentence[256];
    const wchar_t vowels[] = L"аеєиіїоуюяаеіouy";
    const wchar_t consonants[] = L"бвгґджзйклмнпрстфхцчшщbcdfghjklmnpqrstvwxz";
    int vowels_count = 0;
    int consonants_count = 0;
    wscanf(L"%255l[^\n]", sentence);
    for (int i = 0; sentence[i] != L'\0'; i++) {
        wchar_t ch = towlower(sentence[i]);
        if (wcschr(vowels, ch)) { // wcschr шукає збіги в списку
            vowels_count++;
        } else if (wcschr(consonants, ch)) {
            consonants_count++;
        }
    }
    wprintf(L"Голосні: %d\n", vowels_count);
    wprintf(L"Приголосні: %d\n", consonants_count);
    return 0;
}
```

Рис. 1. Код task1.c — підрахунок голосних і приголосних

2.2. Результат виконання

```
└─>> ./task1
Введіть речення: введіть речення
Голосні: 5
Приголосні: 8
arch in  /2026_OP_FEP_13/Martsinko/Lab6
```

Рис. 2. Вивід task1: «введіть речення» → Голосні: 5, Приголосні: 8

Для фрази «введіть речення» програма коректно нарахувала 5 голосних і 8 приголосних. Без setlocale і wchar_t введення кирилиці в Linux ігнорується — тому в коді використано широкі символи та %l[...].

3. Завдання 2. Сума перших 100 чисел без номера в журналі та 33 (while + continue)

У циклі while від 1 до 100 накопичується сума. Числа 33 і введене користувачем (порядковий номер у журналі) пропускаються оператором continue.

3.1. Текст програми (task2.c)

```c
#include <stdio.h>

int main() {
    int i = 1;
    int result = 0;
    int num;
    printf("Введіть число: ");
    scanf("%d", &num);

    while (i <= 100) {
        if (i == 33 || i == num) {
            i++;
            continue;
        }
        result += i;
        i++;
    }
    printf("%d", result);
}
```

Рис. 3. Код task2.c — while, continue

3.2. Результат виконання

```
arch in …/2026_OP_FEP_13/Martsinko/Lab6
└─>> ./task2
Введіть число: 7
5010
```

Рис. 4. Вивід task2: номер 7 → сума 5010

Сума чисел від 1 до 100 дорівнює 5050. Виключено 33 і 7: 5050 − 33 − 7 = 5010 — результат збігається з виводом програми.

4. Завдання 3. N чисел: min, max, sum, avg (цикл for)

Користувач задає N ∈ [7; 12]. Далі в циклі for вводиться N дійсних чисел; обчислюються мінімум, максимум, сума та середнє арифметичне. min і max ініціалізуються першим введеним значенням, щоб уникнути читання неініціалізованої пам’яті.

4.1. Текст програми (task3.c)

```c
#include <stdio.h>

int main() {
    int n;
    printf("Введіть число від 7 до 12: ");
    scanf("%d", &n);
    if(n > 12 || n < 7){
        printf("Введіть коректне число");
        return 1;
    }
    double array[n];

    double min;
    double max;
    double sum;

    for (int i = 0; i < n; i++){
        printf("Введіть число: ");
        scanf("%lf", &array[i]);
        if (i == 0) {
            min = array[i];
            max = array[i];
        } else {
            if (array[i] < min)
                min = array[i];
            if (array[i] > max)
                max = array[i];
        }
        sum += array[i];
    }
    double avg = sum / n;

    printf("Min: %lf, Max: %lf, Sum: %lf, Avg: %lf\n", min, max, sum, avg);

    return 0;


}
```

Рис. 5. Код task3.c

4.2. Результат виконання

```
arch in …/2026_OP_FEP_13/Martsinko/Lab6
└─>> ./task3
Введіть число від 7 до 12: 8
Введіть число: 3
Введіть число: 6
Введіть число: 3
Введіть число: 8
Введіть число: 3
Введіть число: 9
Введіть число: 3
Введіть число: 6
Min: 3.000000, Max: 9.000000, Sum: 41.000000, Avg: 5.125000
```

Рис. 6. Вивід task3: N = 8 → Min 3, Max 9, Sum 41, Avg 5.125

Для набору 3, 6, 3, 8, 3, 9, 3, 6: min = 3, max = 9, sum = 41, avg = 5,125. Перевірка діапазону N і ініціалізація min/max першим елементом забезпечують коректну роботу.

Висновки

Під час виконання лабораторної роботи було створено три програми та на практиці розібрано кілька важливих нюансів мови С. У першому завданні з'ясувалося, що робота з кирилицею в Linux потребує обов'язкового налаштування локалі й використання широких

символів wchar_t разом із правильним специфікатором %l[...], інакше введення просто ігнорується програмою. У другому завданні з циклом while виявилася важливість правильної граничної умови, оскільки через неточне порівняння останнє число випадало з підрахунку суми. У третьому завданні найбільше труднощів виникло з пошуком мінімуму та максимуму: спроба взяти значення з масиву до фактичного введення призвела до читання неініціалізованої пам'яті зі стеку, через що мінімум завис на нулі, і це було виправлено ініціалізацією змінних першим реальним числом. Усі програми налагоджено, помилки усунено, результати обчислень повністю відповідають поставленим вимогам.